day, because the syntax is correct but the semantics are not!

```
ghci> Date 2014 "September" 3
Date 2014 "September" 3
```

You can also use the record syntax that can give you helper functions:

```
data Date = Date { day :: Int
                 , month :: String
                 , year :: Int
                 }
```

This gives you three helper functions to retrieve the day, month and year from a 'Date'.

```
ghci> let d = Date {day = 14, month = "September", year =
2014}
ghci> day d
14
ghci> month d
"September"
```

You can also make data type definitions more explicit with types:

```
data Date = Date Day Month Year deriving (Show)

type Year = Int

type Day = Int

data Month = January
     | February
     | March
     | April
     | May
     | June
     | July
     | August
     | September
     | October
     | November
     | December
     deriving (Show)
```

Loading the above in GHCi, you can use:

```
ghci> Date 3 September 2014
Date 3 September 2014
```

To support printing the date in a specific format, you can implement an instance for the 'Show' type class. You can also add a check to ensure that the day is within a range, and the year and day cannot be swapped:

```
instance Show Date where
    show (Date d m y)
         | d > 0 && d <= 31 = (show d ++ " " ++ show m ++ " "
++ show y)
         | otherwise = error "Invalid day"
```

Load the code in GHCi, and run the following:

```
ghci> show (Date 3 September 2014)
"3 September 2014"
ghci> show (Date 2014 September 2)
"*** Exception: Invalid day
```

Suppose you wish to support different Gregorian date formats, you can define a data type 'GregorianDate' as follows:

```
data GregorianDate = DMY Day Month Year | YMD Year Month Day
```

You can also define your own type classes for functions that define their own behaviour. For example, if you wish to dump the output of a date that is separated by dashes, you can write a 'Dashed' class with a dash function.

```
class Dashed a where
    dash :: a -> String

instance Dashed Date where
    dash (Date d m y) = show d ++ "-" ++ show m ++ "-" ++
show y
```

Testing the above in GHCi will give the following output:

```
ghci> dash (Date 14 September 2014)
"14-September-2014"
```

Haskell allows you to define recursive data types also. A parametrised list is defined as:

```
data List a = Empty | Cons a (List a) deriving (Show)
```

Lists for the above definition can be created in GHCi, using the following commands:

```
ghci> Empty
Empty
ghci> (Cons 3 (Cons 2 (Cons 1 Empty)))
(Cons 3 (Cons 2 (Cons 1 Empty)))
```

END

By: Shakthi Kannan

The author is a free software enthusiast and blogs at *shakthimaan.com*.